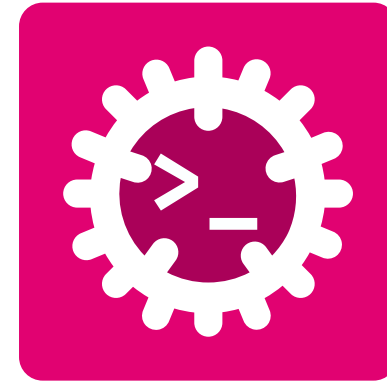


Advanced Programming

Introduction



Silvan Zahno

University of Applied Sciences Western Switzerland - HES-SO Valais Wallis
Systems Engineering
Infotronics

2026-08-07 - v0.1.0



Contents

Introduction	4
Modules	5
Goal of the course AProg and ProgA	7
Grades and Expectations	8
Commit Messages	10
Generative AI	11
Generative AI utilisation in Advanced Programming (AProg)	12
AI Tutor - the rules	13
Organisation	14
Documents and Information	15
Where in the Computer Abstraction are we located?	16
What programming languages do you know?	17
Why Rust?	18
Compile and Run	21
What happens when we are not careful?	22



Where does your code run?	23
What memory is your program using?	24
Heap Vs. Stack memory allocation?	25
Memory Fragmentation	27
References	28
Rust Project and Toolchain	30
Rust Projects	31
Binary vs Library crate	32
Hello World in Rust	33
Rust Project Structure Example	34
Rust Toolchain	36
rustup	37
rustc	38
cargo	39
Cargo.toml	40
Binary vs Library	41



Crates ecosystem 42

RustDoc 43

Glossary & Bibliography 44



Introduction

The Good, The Bad and The Ugly



Resp.	No	Abrév.	Modules Module Cours Vorlesungen	Credits	Heures par semestre Stunden pro Semester					
					1	2	3	4	5	6
anx	I3.1	EmS1	Embedded Systems 1	11			165			
sap	I3.11	EmP	Embedded Programming				75			
anx	I3.12	ElNB	Basic Electronic Circuits				90			
zas	I3.2	SWD1	Software Development 1	7			105			
zas	I3.21	AProG	Advanced Programming				75			
zas	I3.22	ProGA	Applied Programming				30			
mei	I4.1	OT1	Operational Technology 1	6				150		
mei	I4.11	CNet	Communication Networks					75		
anx	I4.12	ElNC	Circuit Design					75		
zas	I4.2	SWD2	Software Development 2	4				75		
zas	I4.21	MDSW	Model-Driven Software Engineering					30		
cfr	I4.22	AML	Applied Machine Learning					45		
mei	I5.1	EmS2	Embedded Systems 2	10					180	
anx	I5.11	AEC	Advanced Electronic Circuits						90	
mei	I5.12	ESE	Embedded Systems Engineering						90	
sti	I5.2	OT2	Operational Technology 2	10					150	
day	I5.21	RSD	Reliable Software Development						90	
sti	I5.22	IIoT	Industrial Internet of Things						60	
mei	I6.1	EmS3	Embedded Systems 3	6						160
gua	I6.11	SoC	System-on-Chip							80
mei	I6.12	RTOS	Real-Time OS							80
anx	I6.2	Data	Data	4						40
anx	I6.21	DAq	Data Acquisition							40
nir	I6.22	SProc	Signal Processing							(30)



Resp.	No	Abrév.	Modules Module Cours Vorlesungen	Credits	Heures par semestre Stunden pro Semester					
					1	2	3	4	5	6
anx	I3.1	EmS1	Infotonics	11			165			
sap	I3.11	EmP					75			
anx	I3.12	ElNB					90			
zas	I3.2	SWD1		7			105			
zas	I3.21	AProG					75			
zas	I3.22	ProGA					30			
mei	I4.1	OT1		6				150		
mei	I4.11	CNet						75		
anx	I4.12	ElNC						75		
zas	I4.2	SWD2		4				75		
zas	I4.21	MDSW						30		
cfr	I4.22	AML						45		
mei	I5.1	EmS2		10					180	
anx	I5.11	AEC							90	
mei	I5.12	ESE							90	
sti	I5.2	OT2		10					150	
day	I5.21	RSD							90	
sti	I5.22	IIoT							60	
mei	I6.1	EmS3		6						160
gua	I6.11	SoC								80
mei	I6.12	RTOS								80
anx	I6.2	Data		4						40
anx	I6.21	DAq								40
nir	I6.22	SProc								(30)



AProg

Develop the ability to design, implement, test, and maintain robust software systems by applying modern programming paradigms, software design principles, concurrency, and software quality practices.

Applied Programming (ProgA)

Apply software engineering methods to develop a complete software system from analysis and design to implementation, testing, and delivery.



Course Structure

- 6h/week for week 1-7, 8h/week for week 8-15
- Project last 4 weeks @ 8h/week

Grading

Nbr	Course	Weighting				
		Exam	Labo/Exercises	Project	Semesterexam	Course
I3.21	Advanced Programming	-	1	-	2	2
I3.22	Applied Programming	-	-	1	-	1

Expectations

- Attend and actively participate in lectures and labs
- Complete exercises, labs, and project milestones
- Work independently outside class hours
- **Read and Write** a lot of code



Evaluation Criteria for Labs, Exercises and Project

Git workflow - required for all deliverables:

```
# Work and commit regularly during the session
git add .
git commit -m "feat: exercise 3 - temperature
converter"

# Mark completion with a tag after each series /
labo
git tag exercises-03
git push && git push --tags
```

- Commits show **progress** and **effort**
- Tags mark **submission points** - missing tag = not submitted
- Commit history is part of the evaluation

Exercises

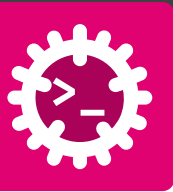
- Number of exercises **completed**
- Number of **unit tests** passing
- Code quality and structure

Labos

- One labo chosen **at random** for detailed review
- Code quality and structure
- Comments and naming

Project

- Exhaustive README.md
- Clean, documented code
- Working features
- Short written report

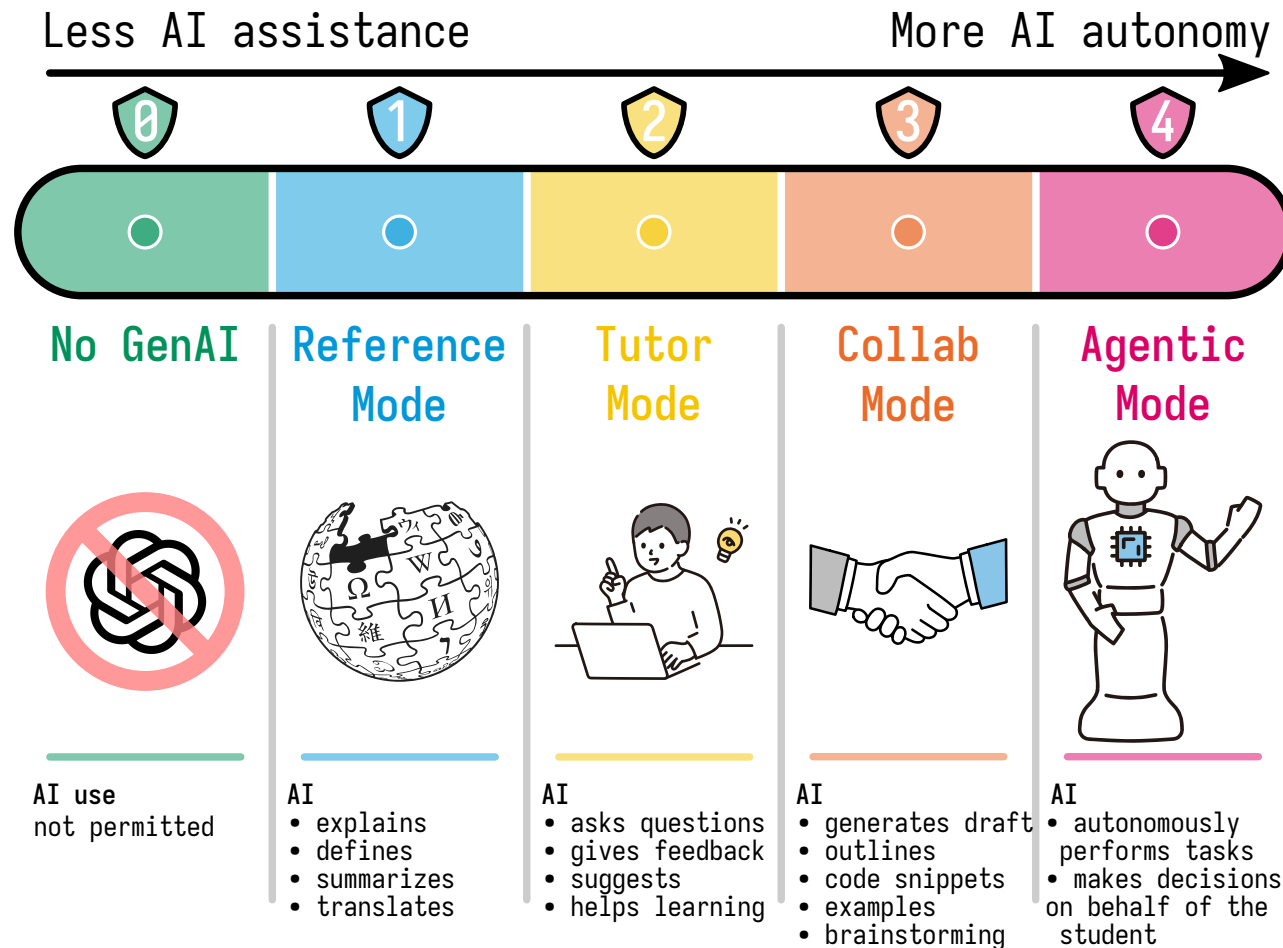


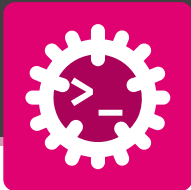
Conventional Commits

Format: `<type>(<scope>): <description>`

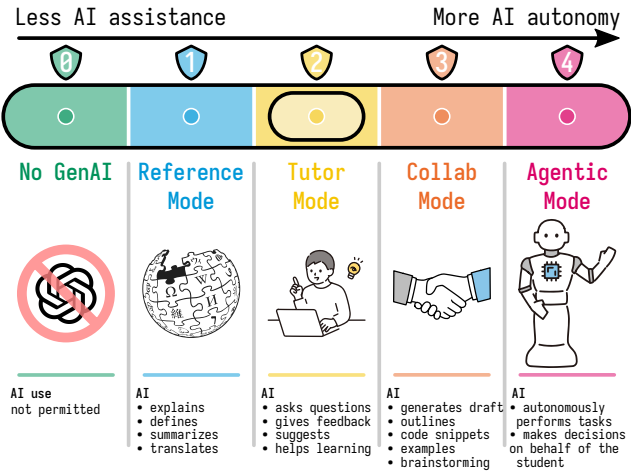
```
feat(ex02): implement temperature converter
fix(ex03): correct overflow in binary inspector
test(ex04): add unit tests for distance converter
refactor(labo1): extract knight move validation
docs(project): write README with setup instructions
chore(project): add justfile build recipe
```

Type	When to use
feat	new feature or behaviour
fix	bug fix
docs	documentation only
style	formatting, no logic change
refactor	improve code without changing behaviour
test	add or update tests
chore	build, CI, tooling
perf	performance improvement
revert	undo a previous commit

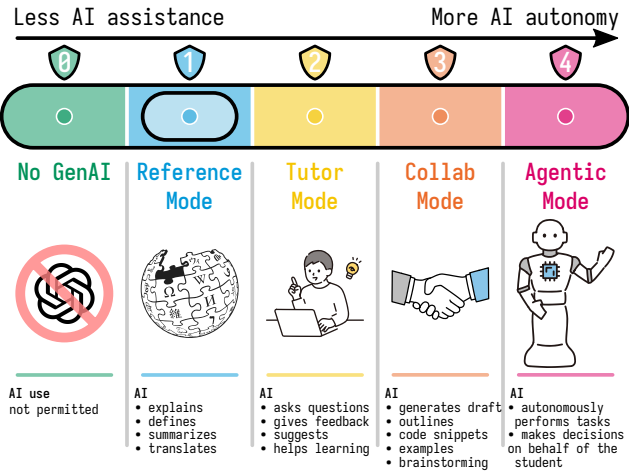




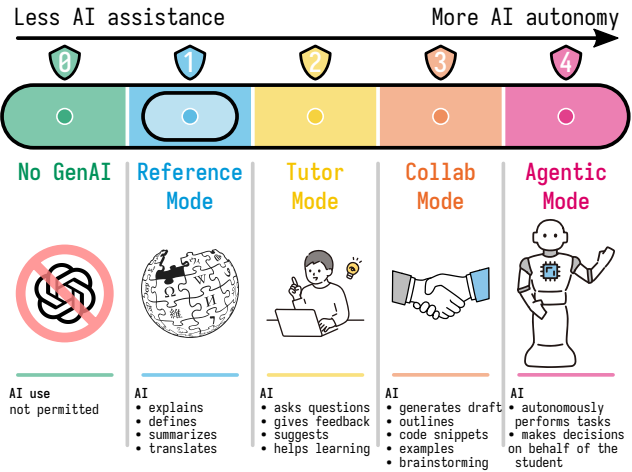
Course



Exercises & Labo



Project





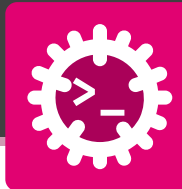
Dedicated **AI prompt** for each level are provided (Course / Exercises & Labo / Project).

1. **Use the given prompt** - it is **mandatory** for **all** AI interactions in this course.
2. **Log every interaction** in LOG.md in your **Labs repo**.
3. **AI helps you learn** - it does **not** do the work for you.
4. **Ask for explanations** - not hints or finished solutions.



Exercises, labs and project are **automatically checked** for AI-generated content.

Any detected AI usage that does **not follow these guidelines** will be **penalized**.



Zahno Silvan (ZaS)

🏢 ENG.23.N312

✉️ silvan.zahno@hevs.ch

☎️ +41 58 606 88 07

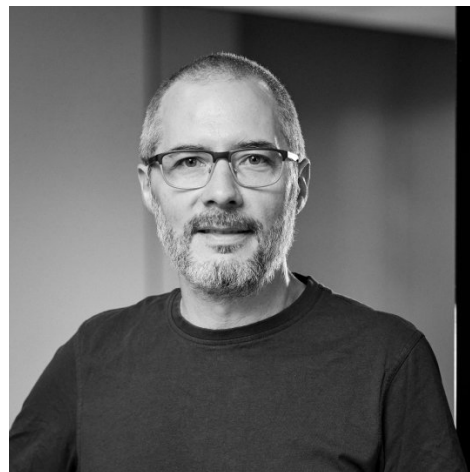


Roudy Dagher (DaY)

🏢 ENG.23.N309

✉️ roudy.dagher@hevs.ch

☎️ +41 58 606 95 33

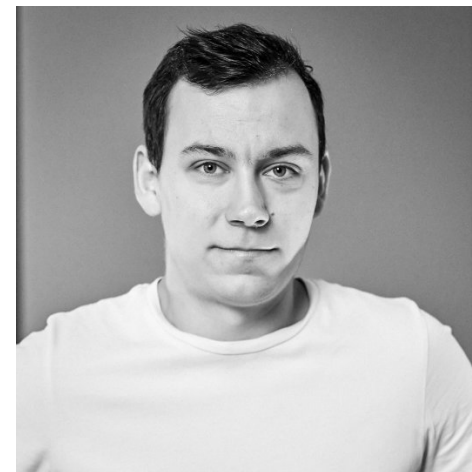


Thomas Sterren (STi)

🏢 ENG.23.N306

✉️ thomas.sterren@hevs.ch

☎️ +41 79 578 87 25



Rémy Borgeat (BoY)

🏢 ENG.23.N313

✉️ remy.borgeat@hevs.ch

☎️ +41 79 578 92 20



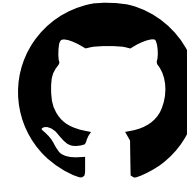
Teams



Name: [SYND-SwD](#)

Code: **efjvezh**

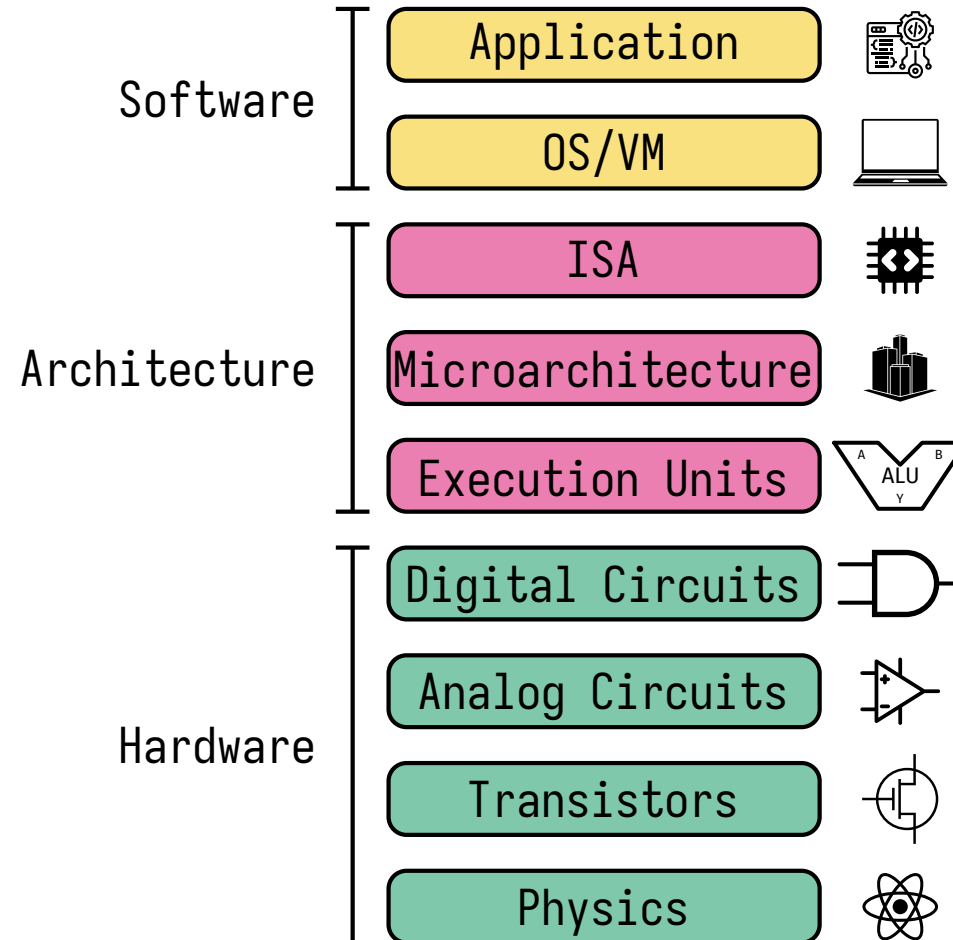
Github



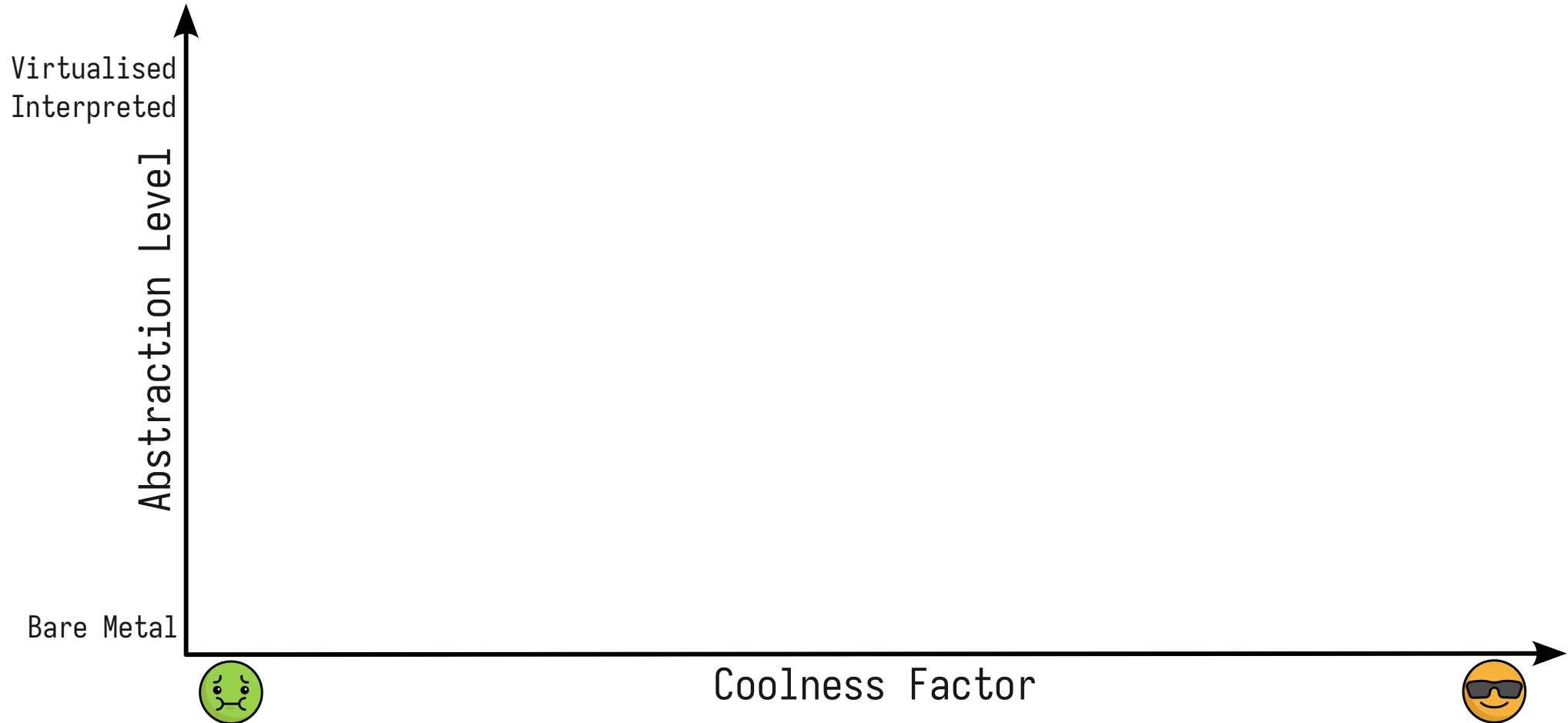
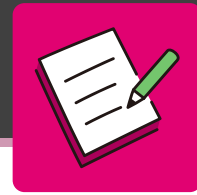
Organization for course documents:
[github hei-synd-swd](#)

Organization for course projects/labs/exercises:
[github hei-synd-swd-stud](#)

Where in the Computer Abstraction are we located?



What programming languages do you know?

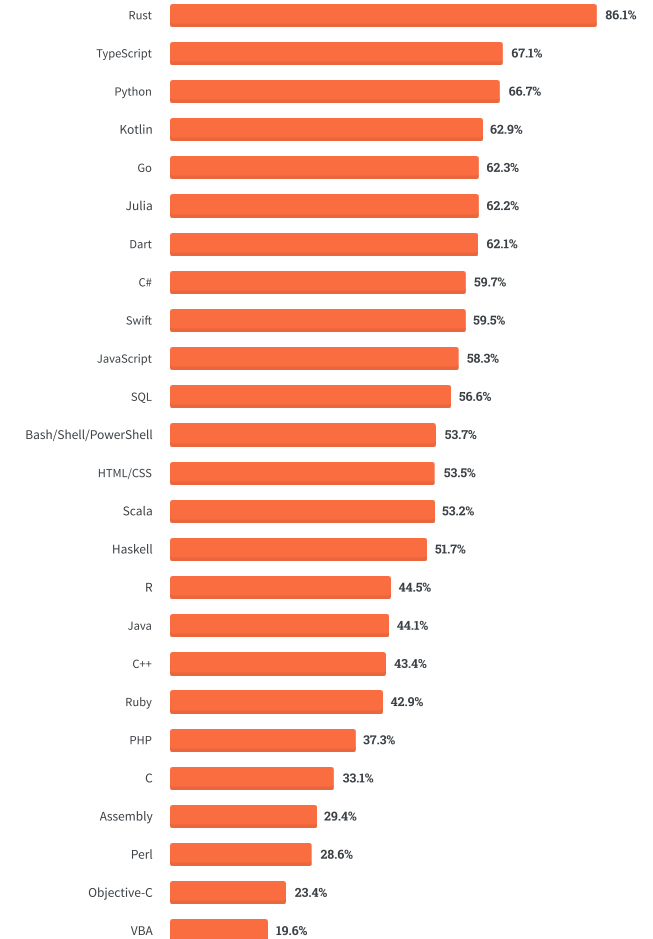


Why Rust?



Most admired language by developers since 2016 according to the [StackOverflow Developer Survey](#)

It is used in the *Linux Kernel* for graphics drivers, at Microsoft for the *Windows* Kernel, at Amazon for *Amazon Web Services (AWS)* for backend services, at Google for *Android* and at *Cloudflare* for their edge network.



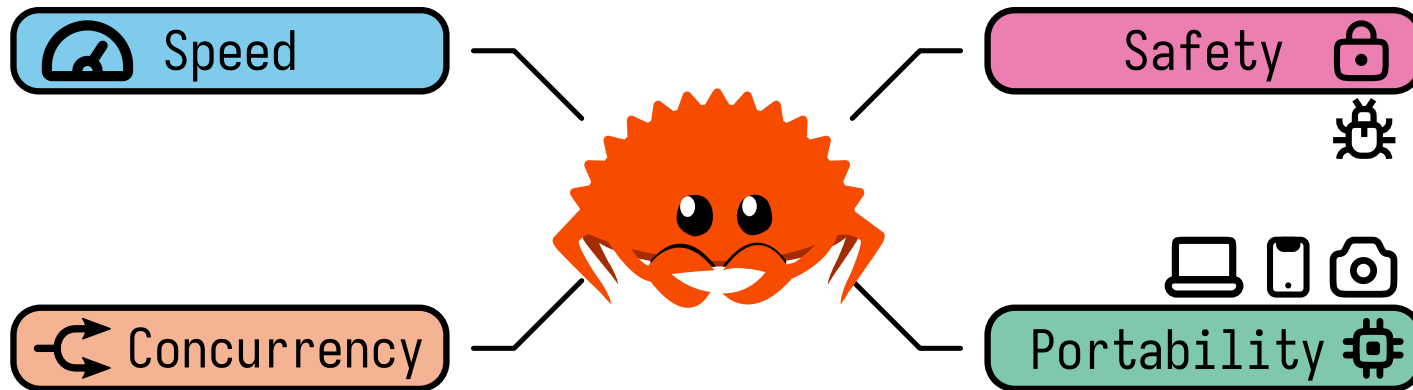


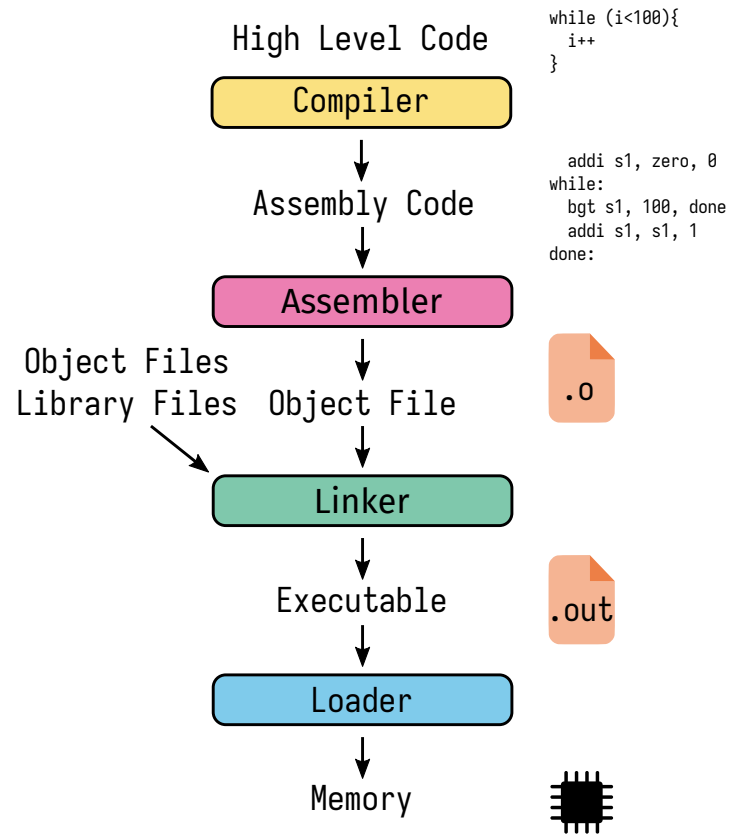
Energy efficient, fast and memory efficient.

Total					
	Energy		Time		Mb
(c) C	1.00	(c) C	1.00	(c) Pascal	1.00
(c) Rust	1.03	(c) Rust	1.04	(c) Go	1.05
(c) C++	1.34	(c) C++	1.56	(c) C	1.17
(c) Ada	1.70	(c) Ada	1.85	(c) Fortran	1.24
(v) Java	1.98	(v) Java	1.89	(c) C++	1.34
(c) Pascal	2.14	(c) Chapel	2.14	(c) Ada	1.47
(c) Chapel	2.18	(c) Go	2.83	(c) Rust	1.54
(v) Lisp	2.27	(c) Pascal	3.02	(v) Lisp	1.92
(c) Ocaml	2.40	(c) Ocaml	3.09	(c) Haskell	2.45
(c) Fortran	2.52	(v) C#	3.14	(i) PHP	2.57
(c) Swift	2.79	(v) Lisp	3.40	(c) Swift	2.71
(c) Haskell	3.10	(c) Haskell	3.55	(i) Python	2.80
(v) C#	3.14	(c) Swift	4.20	(c) Ocaml	2.82
(c) Go	3.23	(c) Fortran	4.20	(v) C#	2.85
(i) Dart	3.83	(v) F#	6.30	(i) Hack	3.34
(v) F#	4.13	(i) JavaScript	6.52	(v) Racket	3.52
(i) JavaScript	4.45	(i) Dart	6.67	(i) Ruby	3.97
(v) Racket	7.91	(v) Racket	11.27	(c) Chapel	4.00
(i) TypeScript	21.50	(i) Hack	26.99	(v) F#	4.25
(i) Hack	24.02	(i) PHP	27.64	(i) JavaScript	4.59
(i) PHP	29.30	(v) Erlang	36.71	(i) TypeScript	4.69
(v) Erlang	42.23	(i) Jruby	43.44	(v) Java	6.01
(i) Lua	45.98	(i) TypeScript	46.20	(i) Perl	6.62
(i) Jruby	46.54	(i) Ruby	59.34	(i) Lua	6.72
(i) Ruby	69.91	(i) Perl	65.79	(v) Erlang	7.20
(i) Python	75.88	(i) Python	71.90	(i) Dart	8.64
(i) Perl	79.58	(i) Lua	82.91	(i) Jruby	19.84



Rust Concepts





```
$ rustc -o hello -C opt-level=3 main.rs
```

Bash



Ariane 5 Flight 501 failure 1997

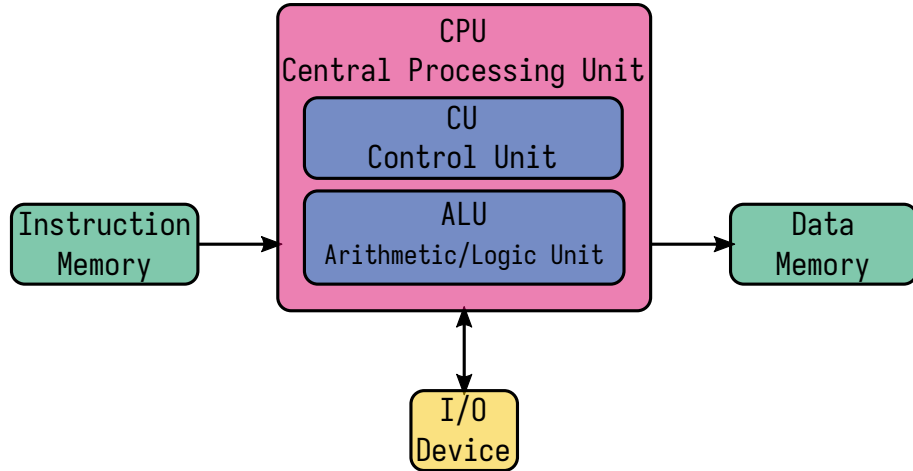


Data conversion from 64-bit floating point to 16-bit signed integer value

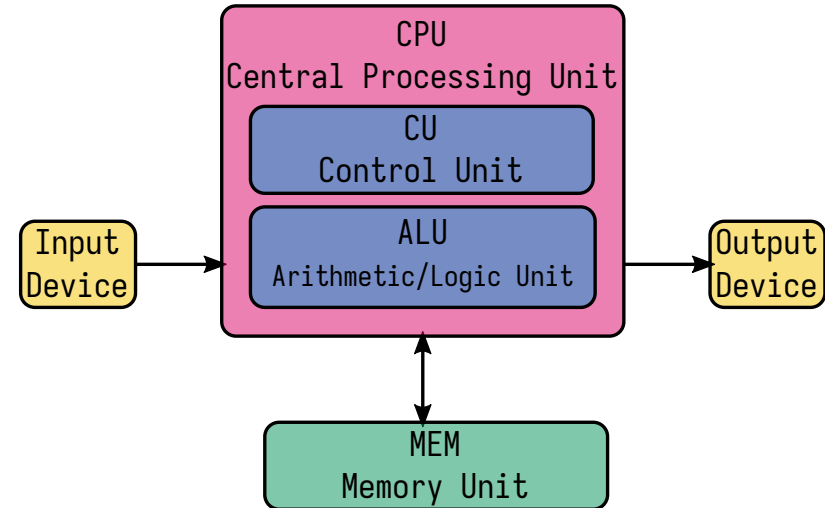
Where does your code run?



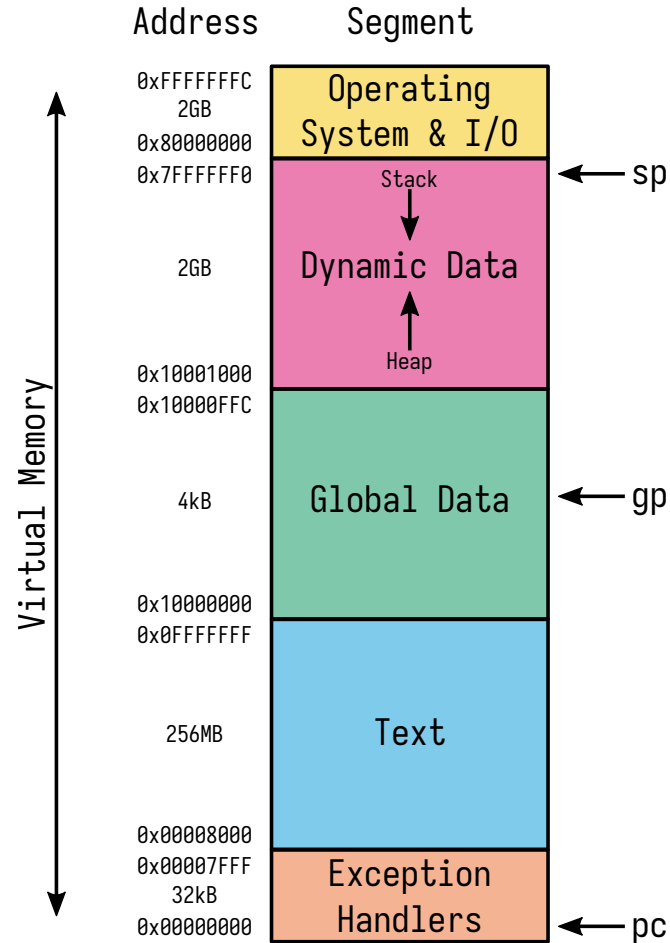
Harvard Architecture



Von Neumann Architecture



What memory is your program using?





Both live in **Random Access Memory (RAM)**, but are managed differently.

The **Stack** holds each function's **local variables**: it grows when a function is **called** and shrinks when it **returns**, so its memory is reserved and freed **automatically**.

The **Heap** is a large pool for values whose **size or lifetime** is only known at **runtime**, allocated and released **on demand**.

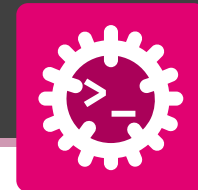
Stack

- ✓ Fast, predictable (just move a pointer)
- ✓ Freed **automatically** at scope end (**Resource Acquisition Is Initialization (RAII)**)
- ✓ No fragmentation
- ✗ Fixed size, reserved **up front**
- ✗ Overflow **crashes** the program

Heap

- ✓ Flexible - **dynamic** size and lifetime
- ✓ Allows **shared ownership**
- ✗ Slower allocation at runtime
- ✗ Manual / managed cleanup - can **leak**
- ✗ **Fragmentation** and out-of-memory over time

Heap Vs. Stack memory allocation?



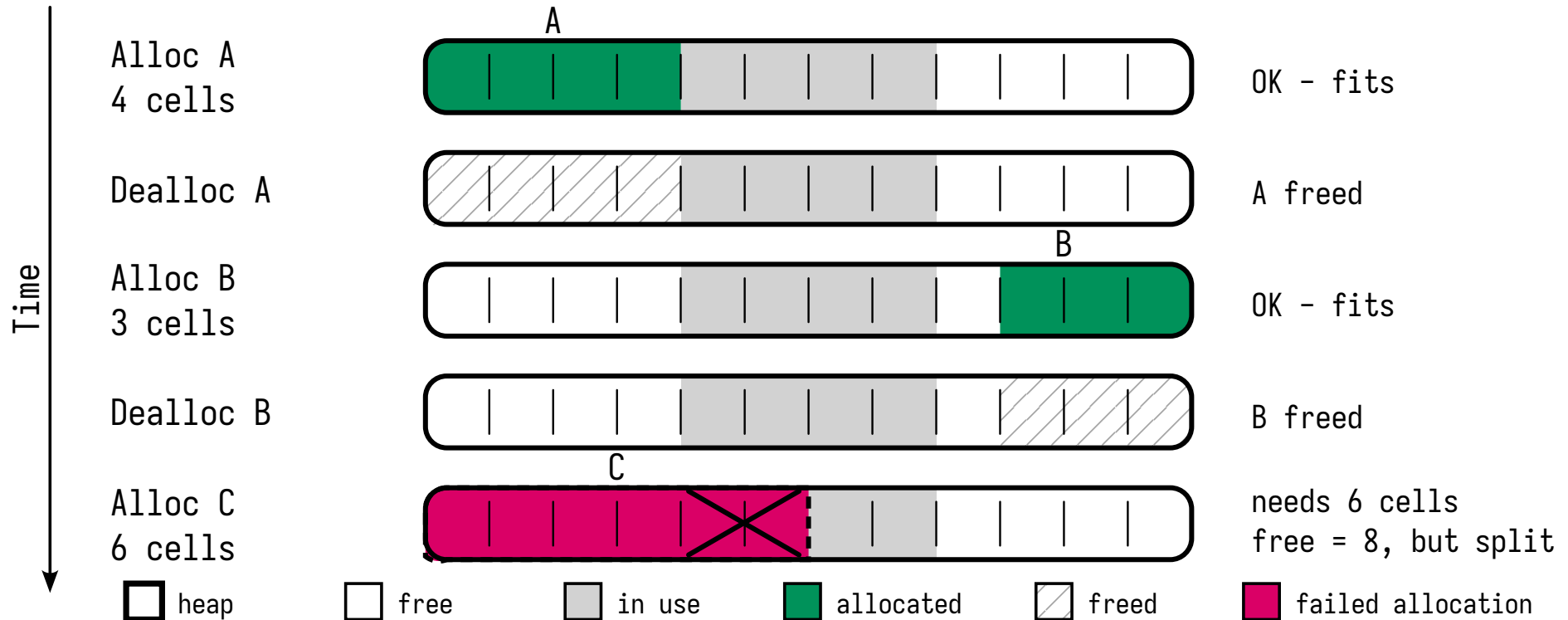
Typical sizes differ by **orders of magnitude** across platforms:

Platform	RAM	Stack (default)	Heap
Desktop / Laptop	8-32 GB	8 MB / thread	up to RAM (GBs)
Raspberry Pi	1-8 GB	8 MB / thread	up to RAM (GBs)
STM32 (Cortex-M)	20 KB SRAM	1-4 KB	0 - a few KB



On a small **STM32**, **Stack** and **Heap** share the **same** 20 KB - many embedded programs use **no heap at all**.

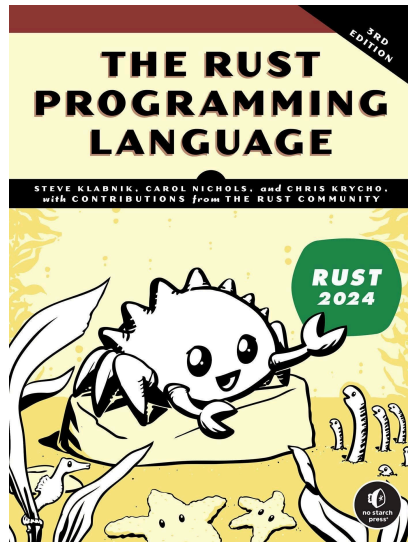
Memory Fragmentation



Fragmentation: the free space is enough, but it is split into non-contiguous holes.



Many additional resources are available. Our library has a good collection of books about programming, software engineering and Rust. Here are some of the most relevant ones for this course:



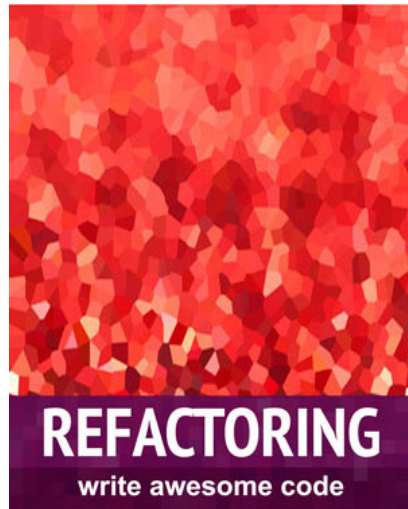
[1]



[2]



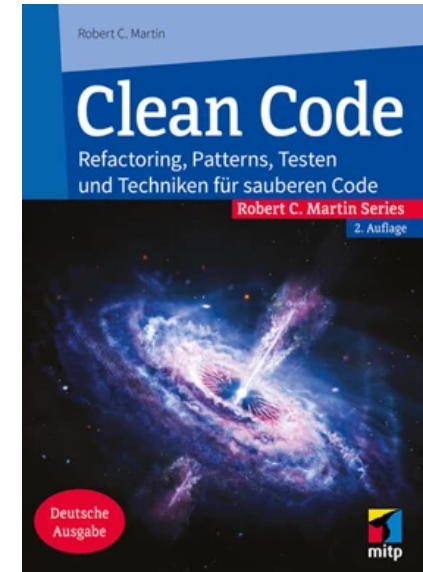
[3]



[4]



[4]



[5]

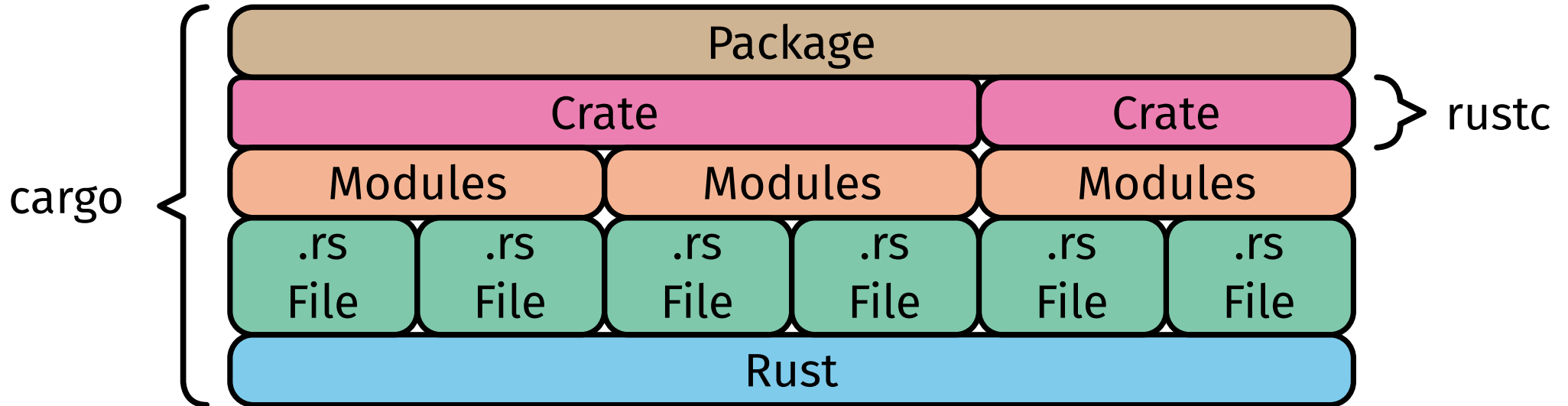


Rust Project and Toolchain

Getting started with Rust projects and toolchain



Package $\Rightarrow$ Crate $\Rightarrow$ Module $\Rightarrow$ Source File





Binary crate

Compiles to an **executable**. Must have a `fn main()` entry point.

```
my_app/
├── Cargo.toml
└── src/
    ├── main.rs      ← entry point
    └── bin/
        └── tool.rs  ← extra binary
```

```
cargo new my_app
cargo run
cargo run --bin tool
```

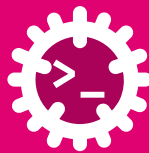
Library crate

Reusable crate (`lib.rs`). No `main()`.

```
my_lib/
├── Cargo.toml
└── src/
    └── lib.rs      ← crate root
```

```
cargo new --lib my_lib
cargo build
cargo test
```

```
# in consumer's Cargo.toml
[dependencies]
my_lib = { path = "../my_lib" }
```



Create and run

```
$ cargo new hello_world
$ cd hello_world
$ cargo run
```

```
hello_world/
├── Cargo.toml
└── src/
    └── main.rs
```

Cargo.toml

```
[package]
name     = "hello_world"
version  = "0.1.0"
edition = "2024"
```

src/main.rs

```
fn main() {
    println!("Hello, world!");
}
```

Output

```
Compiling hello_world v0.1.0
Finished dev in 0.42s
Running `target/debug/hello_world`
Hello, world!
```



A single package can expose both a library (src/lib.rs) and one or more binaries (src/main.rs) at the same time.

```
my_project/  
├── Cargo.toml  
├── Cargo.lock  
└── src/  
    ├── main.rs  
    ├── lib.rs  
    ├── config.rs  
    └── network/  
        ├── mod.rs  
        └── client.rs
```

Cargo.toml

```
[package]  
name      = "network-project"  
version   = "0.1.0"  
edition   = "2024"  
  
[dependencies]  
serde     = { version = "1", features = ["derive"] }  
log        = "0.4"  
request    = "0.12"
```

src/lib.rs - declare modules

```
pub mod config;  
pub mod network;
```



src/network/mod.rs

```
pub mod client;
```

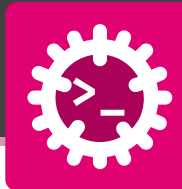
src/network/client.rs

```
use serde::Deserialize;    // external crate
use crate::config::Config; // internal module
#[derive(Deserialize)]
pub struct Response { pub status: u32 }
pub fn fetch(cfg: &Config) → Response { todo!() }
```

src/main.rs

```
use my_project::network::client;
use my_project::config::Config;

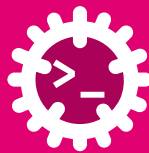
fn main() {
    let cfg = Config::load();
    let res = client::fetch(&cfg);
}
```



The Rust toolchain is a set of tools to write, compile, and manage Rust projects:

- **rustup** - toolchain installer and version manager
- **rustc** - the Rust compiler
- **cargo** - build system and package manager
- **rustdoc** - documentation generator
- **rustfmt** - code formatter
- **clippy** - linter





rustup installs and manages Rust toolchain versions.

Install

```
curl --proto '=https' --tlsv1.2 \  
-sSf https://sh.rustup.rs | sh
```

Common commands

```
rustup update           # update toolchain  
rustup show             # active toolchain  
rustup target add ...   # cross-  
compilation  
rustup component add rustfmt clippy # add components
```

Channels

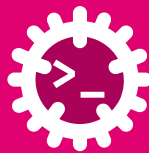
- **stable** - recommended for production
- **beta** - next stable release candidate
- **nightly** - latest features, may be unstable

```
rustup toolchain install nightly  
rustup default stable
```

rust-toolchain.toml

Pin the toolchain for a project:

```
[toolchain]  
channel = "stable"  
components = ["rustfmt", "clippy"]
```



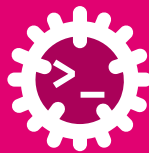
rustc is the Rust compiler. It takes .rs source files and produces a native binary or library.

Usage

```
rustc main.rs                # compile
rustc -o hello main.rs       # named output
rustc --edition 2024 main.rs # compile for a given edition
rustc --explain E0308        # explain error
rustc -o hello -C opt-level=3 main.rs # compile with optimization
rustc -h                     # see help
```



In practice, cargo invokes rustc automatically.

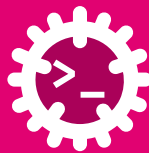


cargo is the Rust build system and package manager.

Most used commands

```
cargo new my_project    # create new project
cargo build              # compile (debug)
cargo build --release    # compile (optimized)
cargo run                # build and run
cargo test              # run tests
cargo check              # type-check, no binary
cargo fmt               # format code
cargo clippy            # run linter
cargo doc --open         # build and open docs
cargo add serde          # add a dependency
cargo update             # update dependencies
```





Every Cargo project has a Cargo.toml manifest at its root:

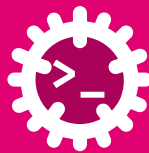
```
[package]
name      = "my_project"
version   = "0.1.0"
edition   = "2024"
authors   = ["Jane Doe <jane@example.com>"]
description = "A sample Cargo project"
license   = "MIT"

[dependencies]
serde = { version = "1", features = ["derive"] }
rand  = "0.8"
hyper = { git = "https://github.com/hyperium/hyper" }

[dev-dependencies]
pretty_assertions = "1"

[profile.release]
opt-level = 3
```

- [package] - metadata (name, version, edition)
- [dependencies] - runtime crates from crates.io or repo links
- [dev-dependencies] - only used for tests/benchmarks
- [profile.*] - compiler flags per build profile



Binary (src/main.rs)

Produces an executable. Entry point is `fn main()`.

```
cargo new my_app
cargo run
```

```
fn main() {
    println!("Hello, world!");
}
```

A project can have **multiple** binaries in `src/bin/`.

Library (src/lib.rs)

Produces a reusable crate. No `main()`. Other projects depend on it via `Cargo.toml`.

```
cargo new --lib my_lib
cargo build
```

```
pub fn add(a: i32, b: i32) → i32 {
    a + b
}
```

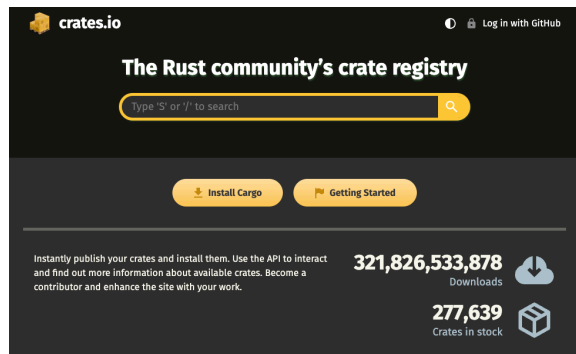
A project can be **both** (lib + bin) at the same time.



<https://crates.io>

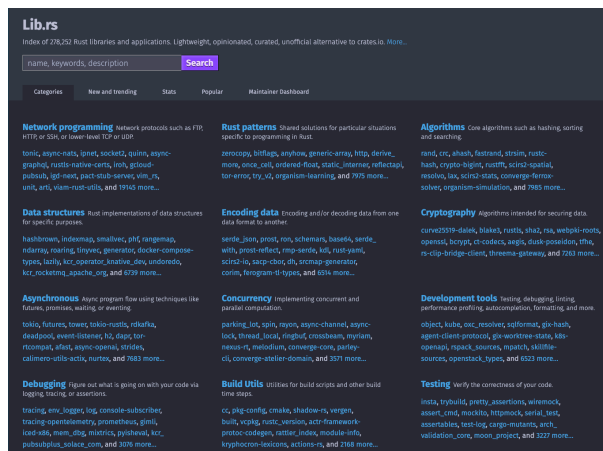


The **official registry** for Rust crates. Publish, search, and manage dependencies. Every crate on crates.io is available for anyone to download and use in their projects.



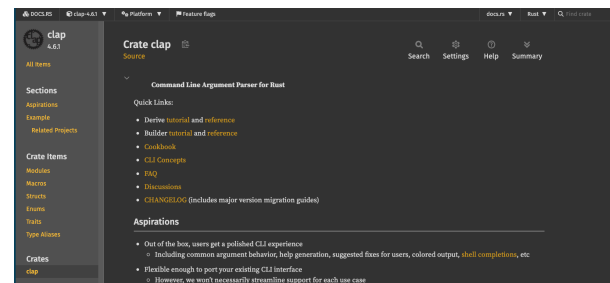
<https://libs.rs>

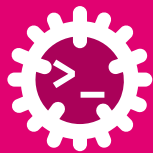
Curated, searchable index of crates with quality scores, recent activity, and dependency graphs. Good for **discovering** well-maintained libraries.



<https://docs.rs>

Automatically generated **Application Programming Interface (API) documentation** for every crate published to crates.io. Built by rustdoc on each release.





rustdoc generates [HyperText Markup Language \(HTML\)](#) documentation from doc comments in source code.

Doc comments

main.rs

```
///! Utility functions.
/// Returns the square of a number.
///
/// # Examples
///
/// ```
/// assert_eq!(square(4), 16);
/// ```
pub fn square(x: i32) → i32 {
    x * x
}
```

Generate and open

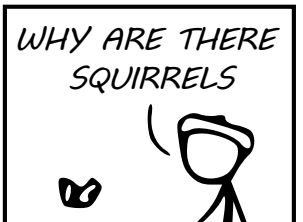
```
cargo doc --open
```

- Module-level docs `///!` - Describe what your crate does
- Function docs `///` - Document every public item
- Code blocks in doc comments are run as **tests** by `cargo test`
- Use proper markdown - Headers, lists, code blocks

Published docs are hosted automatically on [docs.rs](#).

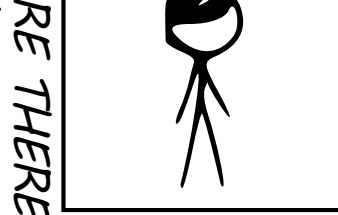
WHY DO WHALES JUMP
WHY ARE THERE MIRRORS ABOVE BEDS
WHY DO I SAY UH
WHY IS SEA SALT BETTER
WHY ARE THERE TREES IN THE MIDDLE OF FIELDS
WHY IS THERE NOT A POKEMON MMO
WHY IS THERE LAUGHING IN TV SHOWS
WHY ARE THERE DOORS ON THE FREEWAY
WHY ARE THERE SO MANY SVCHOST.EXE RUNNING
WHY AREN'T ANY COUNTRIES IN ANTARCTICA
WHY ARE THERE SCARY SOUNDS IN MINECRAFT
WHY IS THERE KICKING IN MY STOMACH
WHY ARE THERE TWO SLASHES AFTER HTTP
WHY ARE THERE CELEBRITIES
WHY DO SNAKES EXIST
WHY DO OYSTERS HAVE PEARLS
WHY ARE DUCKS CALLED DUCKS
WHY DO THEY CALL IT THE CLAP
WHY ARE KYLE AND CARTMAN FRIENDS
WHY IS THERE AN ARROW ON AANG'S HEAD
WHY ARE TEXT MESSAGES BLUE
WHY ARE THERE MUSTACHES ON CLOTHES
WHY WUBA LUBBA DUB DUB MEANING
WHY IS THERE A WHALE AND A POT FALLING
WHY ARE THERE SO MANY BIRDS IN SWISS
WHY IS THERE SO LITTLE RAIN IN WALLIS
WHY IS WALLIS WEATHER FORECAST ALWAYS WRONG
WHY ARE THERE MALE AND FEMALE BIKES

WHY ARE THERE BRIDESMAIDS
WHY DO DYING PEOPLE REACH UP
HOW FAST IS LIGHTSPEED
WHY ARE OLD KLINGONS DIFFERENT

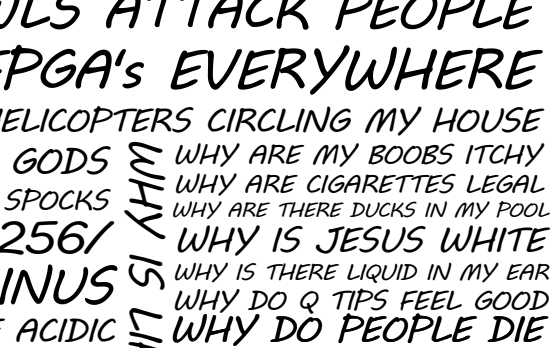


WHY AREN'T THERE DINOSAUR GHOSTS
WHY DO IGUANAS DIE
WHY ARE HATS SO EXPENSIVE
WHY IS THERE CAFFEINE IN MY SHAMPOO
WHY HAVE DINOSAURS NO FUR
WHY AREN'T ECONOMISTS RICH
WHY DO AMERICANS CALL IT SOCCER
WHY ARE MY EARS RINGING
WHY IS 42 THE ANSWER TO EVERYTHING
WHY CAN'T NOBODY ELSE LIFT THORS HAMMER
WHY IS MARVIN ALWAYS SO SAD
WHY IS EARTH TILTED
WHY IS SPACE BLACK
WHY IS OUTER SPACE SO COLD
WHY ARE THERE PYRAMIDS ON THE MOON
WHY IS NASA SHUTTING DOWN
WHY ARE THERE TINY SPIDERS IN MY HOUSE
WHY DO SPIDERS COME INSIDE
WHY ARE THERE HUGE SPIDERS IN MY HOUSE
WHY ARE THERE LOTS OF SPIDERS IN MY HOUSE
WHY ARE THERE SPIDERS IN MY ROOM
WHY ARE THERE SO MANY SPIDERS IN MY ROOM
WHY DO SPYDER BITES ITCH

WHY ARE SWISS AFRAID OF DRAGONS
WHY IS THERE A SWARM OF ANTS
WHY IS THERE PILGRIM
WHY IS THERE LAVA
WHY AREN'T THERE GUNS IN HARRY POTTER
WHAT IS <https://xkcd.com/1256/>
WHY DO THEY SAY T-MINUS
WHY ARE OCEANS BECOMMING MORE ACIDIC

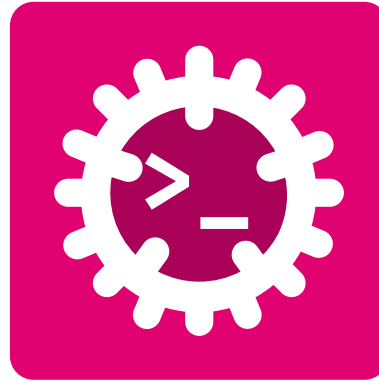


WHY IS HTTPS CROSSED OUT
WHY IS THERE A LINE THROUGH
WHY IS THERE A RED LINE THROUGH HTTPS
WHY IS HTTPS IMPOR
WHY ARE THERE SO MANY CROWS IN ROCHESTER
WHY IS TO BE OR NOT TO BE FL
WHY DO CHILDREN GET CANCER
WHY IS POSEIDON ANGRY WITH ODYSSEUS
WHY IS THERE ICE IN SPACE
WHY IS THERE AN OWL IN MY BACKYARD
WHY IS THERE AN OWL OUTSIDE MY WINDOW
WHY IS THERE AN OWL ON THE DOLLAR BILL
WHY DO OWLS ATTACK PEOPLE
WHY ARE FPGA's EVERYWHERE
WHY ARE THERE HELICOPTERS CIRCLING MY HOUSE
WHY ARE THERE GODS
WHY ARE THERE TWO SPOCKS
WHY ARE MY BOOBS ITCHY
WHY ARE CIGARETTES LEGAL
WHY ARE THERE DUCKS IN MY POOL
WHY IS JESUS WHITE
WHY IS THERE LIQUID IN MY EAR
WHY DO Q TIPS FEEL GOOD
WHY DO PEOPLE DIE



QUESTIONS

CAN BE ASKED BY ANYONE ANYTIME



Glossary & Bibliography



Computer Science

RAM – Random Access Memory: Volatile memory used to store data and programs during execution. [25](#), [26](#), [26](#), [26](#)

Course

AProg – Advanced Programming: Course covering advanced programming concepts using Rust. [1](#), [7](#), [12](#)

ProgA – Applied Programming: Project-oriented course focused on applying programming concepts in practice. [7](#)

Programming

API – Application Programming Interface: A set of rules and interfaces that allows software components to communicate. [42](#)

Heap – Heap: Memory area used for dynamic memory allocation. [25](#), [25](#), [26](#), [26](#)

HTML – HyperText Markup Language: Markup language used to create web pages and generated documentation. [43](#)

RAII – Resource Acquisition Is Initialization: Resource management technique where acquisition and release are tied to object lifetime. [25](#)

Stack – Stack: Memory area used for static memory allocation and function call management. [25](#), [25](#), [26](#), [26](#)

Technology

AWS – Amazon Web Services: Cloud computing platform provided by Amazon. [18](#)



Bibliography

- [1] S. Klabnik, C. Nichols, and C. Krycho, “The Rust Programming Language.”
- [2] J. Blandy, J. Orendorff, and L. F. s Tindall, *Programming Rust: Fast, Safe Systems Development*. O'Reilly Media, Incorporated, 2021.
- [3] H. Collard, *Black Hat Rust*, vol. 1. Amazon, 2025.
- [4] A. Shvets, “Refactoring Guru.” 2026.
- [5] R. C. Martin, *Clean Code: A Handbook of Agile Software Craftsmanship*. Pearson Education, 2008.
- [6] “SVG Repo - Free SVG Vectors and Icons.”
- [7] HEI-VS, “Module Description SWD1.”
- [8] HEI-VS, “Module Description SWD2.”
- [9] R. Pereira *et al.*, “Energy Efficiency across Programming Languages: How Do Energy, Time, and Memory Relate?,” in *Proceedings of the 10th ACM SIGPLAN International Conference on Software Language Engineering*, in SLE 2017. New York, NY, USA: Association for Computing Machinery, Oct. 2017, pp. 256–267. doi: [10.1145/3136014.3136031](https://doi.org/10.1145/3136014.3136031).
- [10] Amazing Info TV, “Ariane 5 Rocket Launch Explosion.” Feb. 2015.